ean universidad

acreditada
en alta calidad
res. N° 023654 del Mineducación
10/12/21, vigencia 10/12/27

PYTHON PARA PROCESAMIENTO, CALIDAD Y AUTOMATIZACIÓN DE DATOS

Sesión 4 · Automatización: funciones, pipelines y reportes

Que el reporte del próximo mes sea un clic

Jawy Andrés Romero Pinto · Viernes 6 de noviembre de 2026 · 08:00 – 10:00 a. m.

SOMOS LA U. DEL
EMPRENDIMIENTO
SOSTENIBLE.

Acreditaciones:

Agenda de hoy

Viernes 6 de noviembre · 08:00 – 10:00 a. m.

08:00	**Bienvenida y reto de ayer**	Las dos reglas que agregaste al catálogo.
08:05	**Qué vale la pena automatizar**	Frecuencia, volumen, riesgo de error y valor del resultado.
08:12	**De celdas a funciones**	Una función, una responsabilidad: parámetros, return y docstring.
08:17	**Muchos archivos**	Nueve meses de aportes con un bucle, y el archivo que llega distinto.
08:27	**Errores y bitácora**	try/except, validar la estructura y dejar rastro de lo que pasó.
08:33	**Resumir y reportar**	Resumen mensual, gráfico guardado y un Excel con varias hojas y formato.
08:40	**Pipeline y producción**	Una función que orquesta todo; script, parámetros y programación.
08:45	**Laboratorio**	Construyes el pipeline de aportes de enero a septiembre (70 min).
09:55	**Cierre y reto**	Lo que te llevas y la preparación del proyecto integrador.

01

De dónde venimos

Ver, corregir, medir y, hoy, repetir sin esfuerzo.

Ver, corregir, medir, repetir

Cada sesión responde una pregunta distinta sobre los mismos datos

Sesión	Pregunta	Qué hicimos	Herramienta
1 · Fundamentos	¿Qué hay en el archivo?	Vimos 39 ciudades, salarios como texto, 90 duplicados	info, value_counts, filtros
2 · Limpieza	¿Cómo lo corrijo?	strip, map, to_numeric, to_datetime, merge	pandas sobre una copia
3 · Calidad	¿Puedo confiar?	Catálogo de reglas, motor, índice y cuarentena	máscaras y funciones
4 · Automatización (hoy)	¿Y el archivo del mes que viene?	Un proceso que lee, valida, resume y reporta solo	funciones, bucles, try, Excel

Automatizar no es hacerlo más rápido una vez: es **no tener que volver a hacerlo**, y que salga igual cada mes.

Qué vale la pena automatizar

Regla práctica: si lo haces más de dos veces a mano, merece un script

Tarea	Frecuencia	Riesgo de error manual	Decisión
Consolidar los archivos de aportes del mes	Mensual	Alto: copiar y pegar 9 archivos	Automatizar
Validar que el aporte sea el 16 % del IBC	Mensual	Alto: miles de filas a ojo	Automatizar
Reporte de recaudo para la dirección	Mensual	Medio: formato y cifras	Automatizar
Análisis de una pregunta nueva de la gerencia	Una vez	Bajo	Notebook
Corregir un dato puntual de un afiliado	Esporádica	Bajo	A mano

Automatiza lo **repetitivo y propenso a errores**; deja el análisis creativo para las personas.

ean universidad

02

De celdas sueltas a funciones

Una función hace una cosa, tiene nombre y se usa muchas veces.

Funciones: una responsabilidad con nombre

Se escribe una vez, se llama nueve veces (o novecientas)

Qué es

Un bloque de código con nombre que recibe datos (**parámetros**), hace una sola tarea y devuelve un resultado (`return`). El texto entre triples comillas (**docstring**) explica qué hace.

Para qué sirve

Que la regla viva en un solo lugar. Si la tasa cambia, se corrige una línea y no nueve copias del mismo bloque. Y cada pieza se puede probar sola.

Analogía con Excel

Una fórmula con nombre que se reutiliza en cualquier hoja, como una función personalizada. Celdas sueltas son fórmulas copiadas a mano: si cambia una, hay que cambiarlas todas.

Cómo manejarlo

Nombre con verbo (`marcar_`, `leer_`, `resumir`). Valores del negocio como constantes arriba o como parámetros con valor por defecto. Siempre `return`. Prueba con un mes antes de usarla con nueve.

Abstracción desde el código

```
TASA = 0.16

def marcar_aporte_incorrecto(df,
        tasa=TASA, tolerancia=1):
    """Marca aportes != ibc * tasa."""
    esperado = (df["ibc"] * tasa).round()
    df["aporte_incorrecto"] = ...
    return df
```

Una función es una caja con entrada y salida: quien la usa no necesita ver adentro, solo saber qué entra y qué sale. Así se construyen procesos grandes con piezas pequeñas que se entienden.

La primera pieza: marcar aportes incorrectos

Probarla con un mes antes de usarla con nueve

[1]
```python
def marcar_aporte_incorrecto(df, tasa=TASA, tolerancia=1):
    """Agrega 'aporte_incorrecto': True si el aporte se aleja
    más de `tolerancia` pesos de ibc * tasa."""
    esperado = (df["ibc"] * tasa).round()
    df["aporte_incorrecto"] = (df["valor_aporte"] - esperado).abs() >
tolerancia
    return df

enero = pd.read_csv(RUTA + "aportes/aportes_2026-01.csv")
enero = marcar_aporte_incorrecto(enero)
print("Aportes incorrectos en enero:", enero["aporte_incorrecto"].sum())
enero[enero["aporte_incorrecto"]][["id_afiliado", "ibc",
"valor_aporte"]].head(3)
```

```
Aportes incorrectos en enero: 17
```

	id_afiliado	ibc	valor_aporte
19	AF100377	1751000	188856
457	AF102556	5527000	545199
622	AF100186	4686000	511030

Qué observar

- `tasa` y `tolerancia` son parámetros con valor por defecto.
- Enero: **17** aportes que no son el 16 % del IBC.
- Se prueba con un archivo; si funciona, sirve para todos.

La misma función se reutiliza más tarde dentro de `marcar_alertas`. Eso es lo que se gana escribiendo funciones: **no se vuelve a escribir la regla**.

03

Procesar muchos archivos

Nueve meses de aportes con un bucle, y el archivo que llega distinto.

Un bucle sobre una lista de archivos

La misma operación para cada mes, sin copiar código

Qué es

Una lista con las rutas de los nueve archivos y un `for` que recorre la lista y hace lo mismo con cada uno. La lista se construye con una **list comprehension** y un f-string.

Para qué sirve

Procesar enero a septiembre, o los meses que lleguen, con el mismo código. El décimo mes no requiere cambiar nada: solo alargar el rango.

Analogía con Excel

Abrir nueve libros, uno por uno, y copiar cada hoja a un consolidado. Aquí la persona no abre nada: el bucle abre, lee y sigue.

Cómo manejarlo

Construye las rutas con una función (`ruta_mes`). `f"{mes:02d}"` pone el cero a la izquierda. En tu PC puedes usar `glob`, pero **glob no funciona con URLs**: en Colab desde GitHub, construye la lista.

Abstracción desde el código

```
def ruta_mes(mes):
    return f"{RUTA}aportes/aportes_2026-{mes:02d}.csv"


rutas = [ruta_mes(m) for m in range(1, 10)]


for ruta in rutas:
    df = pd.read_csv(ruta)
```

El bucle separa el **qué** (leer un archivo) del **cuántos** (la lista). Cambiar cuántos meses se procesan es cambiar un número, no el programa.

La trampa: julio llegó distinto

Todos tienen 6 columnas, menos uno

[2]
```python
def ruta_mes(mes):
    """Ruta del archivo de aportes de un mes de 2026."""
    return f"{RUTA}aportes/aportes_2026-{mes:02d}.csv"

rutas = [ruta_mes(m) for m in range(1, 10)]     # nueve meses

for ruta in rutas:
    df = pd.read_csv(ruta)                       # lectura ingenua
    print(ruta.split("/")[-1], df.shape)
```

```
aportes_2026-01.csv (1770, 6)
aportes_2026-02.csv (1757, 6)
aportes_2026-03.csv (1770, 6)
aportes_2026-04.csv (1775, 6)
aportes_2026-05.csv (1761, 6)
aportes_2026-06.csv (1764, 6)
aportes_2026-07.csv (1785, 1)
aportes_2026-08.csv (1749, 6)
aportes_2026-09.csv (1745, 6)
```

Qué observar

- Ocho meses salen con **6 columnas**.
- `aportes_2026-07.csv` sale con **(1785, 1)**: una sola columna.
- No hubo error: el programa siguió como si nada.

El peor error de un pipeline no es el que se detiene: es **el que sigue funcionando con datos equivocados**. Concatenado sin mirar, julio sería basura silenciosa.

Validar la estructura antes de procesar

Confiar, pero verificar: cada insumo pasa un control

Qué es

Comparar las columnas que llegaron con las que se esperan y devolver las que **faltan**. Y leer de forma robusta: detectar el separador y normalizar los nombres de columna.

Para qué sirve

Que un archivo con otro formato se **detecte** en vez de contaminar el consolidado. Julio trae `;` y encabezados en mayúscula; el próximo mes puede traer otra cosa.

Analogía con Excel

Es la revisión que haces al abrir un archivo nuevo: ¿están las columnas de siempre, en el orden de siempre? Aquí la revisión la hace el programa, cada vez, sin olvidarse.

Cómo manejarlo

`sep=None, engine="python"` detecta coma o punto y coma.
`columns.str.strip().str.lower()` iguala los nombres.
`validar_estructura` devuelve la lista de faltantes: vacía es buena señal.

Abstracción desde el código

```
COLUMNAS = ["id_afiliado", "periodo",
"ibc", ...]

def validar_estructura(df):
    return [c for c in COLUMNAS
            if c not in df.columns]

df = pd.read_csv(ruta, sep=None,
                 engine="python")
```

La lista de columnas esperadas es un **contrato** con quien envía el archivo. El programa no adivina qué llegó: compara contra el contrato y avisa.

Lectura robusta: julio pasa la validación

Detectar el separador y normalizar los nombres

[3]
```python
julio = pd.read_csv(ruta_mes(7))
print(julio.columns.tolist())                    # una sola columna

def leer_aportes(ruta):
    """Lee un archivo de aportes detectando el separador."""
    df = pd.read_csv(ruta, sep=None, engine="python")   # , o ;
    df.columns = df.columns.str.strip().str.lower()      # ID -> id
    return df

def validar_estructura(df, esperadas=COLUMNAS):
    """Columnas esperadas que faltan (vacía = todo bien)."""
    return [c for c in esperadas if c not in df.columns]

print("Ingenua, faltan:", len(validar_estructura(julio)), "de 6")
julio = leer_aportes(ruta_mes(7))
print("Robusta:", julio.shape, "| faltan:", validar_estructura(julio))
```

```
['ID_AFILIADO;PERIODO;IBC;VALOR_APORTE;FECHA_PAGO;OPERADOR']
Ingenua, faltan: 6 de 6
Robusta: (1785, 6) | faltan: []
```

Qué observar

- Lectura ingenua: julio tiene una columna con todo pegado; le faltan **6 de 6**.
- Lectura robusta: **(1785, 6)** y faltan `[]`.
- Las dos funciones son cortas y cada una hace una sola cosa.

`validar_estructura` no arregla nada: **detecta**. La decisión de qué hacer con un archivo malo se toma en el paso siguiente, con `try/except`.

04

Errores y bitácora

Si un archivo falla, el proceso sigue y queda registrado por qué.

Manejo de errores: try / except

Lo que puede fallar se intenta; si falla, se atiende

Qué es

`try` envuelve el código que puede fallar; si falla, Python salta al `except` en vez de detenerse. `raise` lanza un error a propósito cuando una validación no pasa.

Para qué sirve

Que un archivo dañado o ausente no tumbe el proceso de los otros ocho, y que el error **quede anotado** con su tipo, para revisarlo después.

Analogía con Excel

Es SI.ERROR() en una fórmula: si falla, haz otra cosa. La diferencia es que aquí el error no se esconde con un cero: se registra con nombre propio.

Cómo manejarlo

Pon dentro del `try` solo lo que puede fallar. En el `except` **registra siempre** el error. Nunca `except: pass`: esconde el problema y el reporte sale mal sin aviso.

Abstracción desde el código

```
try:
    df = leer_aportes(ruta)
    faltan = validar_estructura(df)
    if faltan:
        raise ValueError(f"faltan {faltan}")
except Exception as e:
    registrar(nombre, f"ERROR: {e}")
```

El error se convierte en un dato: un archivo, una hora y un motivo. Un proceso confiable no es el que nunca falla, sino el que **avisa cuando falla**.

La bitácora: la memoria del proceso

Qué pasó, con qué archivo, a qué hora y con cuántas filas

Qué es

Una lista donde cada paso del proceso agrega una fila: hora, archivo, evento (OK o el error) y filas procesadas. Al final se convierte en un DataFrame y va al reporte.

Para qué sirve

Responder "¿qué pasó?" sin volver a ejecutar nada. Cuando alguien pregunte por una cifra, la bitácora y la columna `archivo_origen` dicen de dónde salió.

Analogía con Excel

La hoja "Control de cambios" que algunos llevan a mano: fecha, qué se hizo, quién. Aquí se llena sola y no se puede olvidar.

Cómo manejarlo

Una función `registrar(...)` que se llama en cada paso importante. Agrega `archivo_origen` a cada fila antes de `pd.concat`. En producción se usa el módulo `logging` y un archivo `.log`.

Abstracción desde el código

```
bitacora = []


def registrar(archivo, evento,
filas=None):
    bitacora.append({"hora": ...,
        "archivo": archivo,
        "evento": evento, "filas": filas})


pd.DataFrame(bitacora)
```

La bitácora es lo primero que se mira cuando algo sale mal. Un proceso sin bitácora obliga a adivinar; uno con bitácora deja la evidencia escrita.

Cargar todo con control de errores

Nueve archivos, un consolidado y un mes que no existe

[4]
```python
from datetime import datetime
bitacora = []

def registrar(archivo, evento, filas=None):
    bitacora.append({"hora": datetime.now().strftime("%H:%M:%S"),
                     "archivo": archivo, "evento": evento, "filas": filas})

def cargar_aportes(meses=range(1, 10)):
    partes = []
    for m in meses:
        nombre = f"aportes_2026-{m:02d}.csv"
        try:
            df = leer_aportes(ruta_mes(m))
            faltan = validar_estructura(df)
            if faltan:
                raise ValueError(f"faltan columnas {faltan}")
            df["archivo_origen"] = nombre          # trazabilidad
            partes.append(df)
            registrar(nombre, "OK", len(df))
        except Exception as e:
            registrar(nombre, f"ERROR: {type(e).__name__}")
    return pd.concat(partes, ignore_index=True)

aportes = cargar_aportes(range(1, 11))          # el mes 10 no existe
print("Consolidado:", aportes.shape)
pd.DataFrame(bitacora).astype({"filas": "Int64"}).tail(4)
```

```
Consolidado: (15876, 7)
```

	hora	archivo	evento	filas
6	21:06:47	aportes_2026-07.csv	OK	1785
7	21:06:47	aportes_2026-08.csv	OK	1749
8	21:06:47	aportes_2026-09.csv	OK	1745
9	21:06:47	aportes_2026-10.csv	ERROR: FileNotFoundError	<NA>

Qué observar

- Consolidado: **15.876 filas** y 7 columnas (6 más `archivo_origen`).
- El mes 10 no existe: la bitácora dice `ERROR: FileNotFoundError`.
- Los otros nueve se procesaron igual.

En Colab, leyendo desde GitHub, el error del mes 10 es `HTTPError` en vez de `FileNotFoundError`. El proceso **se comporta igual**: registra y sigue.

Limpiar y marcar alertas reutilizando funciones

Lo de la Sesión 2 y la Sesión 3, ahora como piezas

[5]
```python
def limpiar_aportes(df):
    antes = len(df)
    df = df.drop_duplicates(subset=COLUMNAS).copy()
    df["fecha_pago"] = pd.to_datetime(df["fecha_pago"], errors="coerce")
    registrar("consolidado", "duplicados eliminados", antes - len(df))
    return df

def marcar_alertas(df):
    df = marcar_aporte_incorrecto(df)          # se reutiliza la función
    limite = pd.to_datetime(df["periodo"] + "-01") + pd.offsets.MonthEnd(2)
    df["pago_tardio"] = df["fecha_pago"] > limite
    df["ibc_bajo_minimo"] = df["ibc"] < SMMLV
    df["tiene_alerta"] = df[["aporte_incorrecto", "pago_tardio",
                             "ibc_bajo_minimo"]].any(axis=1)
    return df

aportes = marcar_alertas(limpiar_aportes(aportes))
print("Filas después de limpiar:", len(aportes))
aportes[["aporte_incorrecto", "pago_tardio", "ibc_bajo_minimo", "tiene_alerta"]].sum()
```

```
Filas después de limpiar: 15831
aporte_incorrecto     154
pago_tardio           850
ibc_bajo_minimo        72
tiene_alerta         1063
Name: None, dtype: int64
```

Qué observar

- 45 duplicados eliminados: quedan **15.831** aportes.
- 154 aportes incorrectos, 850 pagos tardíos, 72 con IBC bajo el mínimo.
- **1.063** aportes con al menos una alerta.

`marcar_alertas` llama a `marcar_aporte_incorrecto`, escrita al comienzo. Cada función nueva se apoya en las anteriores: así crece un proceso sin volverse inmanejable.

ean universidad

05

Resumir y reportar

Lo que la dirección quiere ver cada mes, generado sin tocarlo a mano.

El resumen mensual para la dirección

groupby + agg con nombres, igual que en la Sesión 2

[6]
```python
def resumir(df):
    """Resumen mensual para la dirección."""
    return (df.groupby("periodo")
              .agg(aportantes=("id_afiliado", "nunique"),
                   recaudo=("valor_aporte", "sum"),
                   incorrectos=("aporte_incorrecto", "sum"),
                   tardios=("pago_tardio", "sum"),
                   ibc_bajo_minimo=("ibc_bajo_minimo", "sum"))
              .reset_index())

resumen = resumir(aportes)
print(f"Recaudo total ene-sep: ${resumen['recaudo'].sum():,.0f}")
resumen
```

```
Recaudo total ene-sep: $8,687,101,903
```

	periodo	aportantes	recaudo	incorrectos	tardios	ibc_bajo_minimo
0	2026-01	1765	967824232	17	113	8
1	2026-02	1752	958371317	17	85	8
2	2026-03	1765	971180620	18	88	8
3	2026-04	1770	972857448	17	95	8
4	2026-05	1756	961741356	17	100	8
5	2026-06	1759	964400484	17	95	8
6	2026-07	1780	974807311	17	100	8
7	2026-08	1744	958609318	17	83	8
8	2026-09	1740	957309817	17	91	8

Qué observar

- Una fila por mes: aportantes, recaudo, incorrectos, tardíos e IBC bajo el mínimo.
- Recaudo total enero a septiembre: **$8.687.101.903**.
- El recaudo es estable: entre 957 y 975 millones por mes.

`agg(nombre=("columna", "función"))` deja las columnas con el nombre del reporte. Nadie tiene que renombrar nada después.

Gráfico que se guarda solo

matplotlib dibuja y savefig deja el archivo listo para el reporte

Qué es

`matplotlib` dibuja el gráfico a partir de los datos del resumen y `savefig` lo guarda como imagen PNG. La función recibe el resumen y la ruta, y devuelve el gráfico.

Para qué sirve

Que cada ejecución deje su propio gráfico actualizado, listo para pegarse en el Excel o en una presentación, sin capturar pantalla.

Analogía con Excel

Insertar un gráfico de columnas en Excel y actualizar el rango cada mes. Aquí el rango es el resumen: si cambia el dato, cambia el gráfico.

Cómo manejarlo

Título con unidades, eje en millones, una sola serie y un solo color. Guarda en la carpeta de salidas con un nombre fijo y deja que la función sea quien lo genere siempre.

Abstracción desde el código

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.bar(resumen["periodo"],
       resumen["recaudo"] / 1e6)
ax.set_title("Recaudo mensual (millones)")
fig.savefig("salidas/recaudo.png")
```

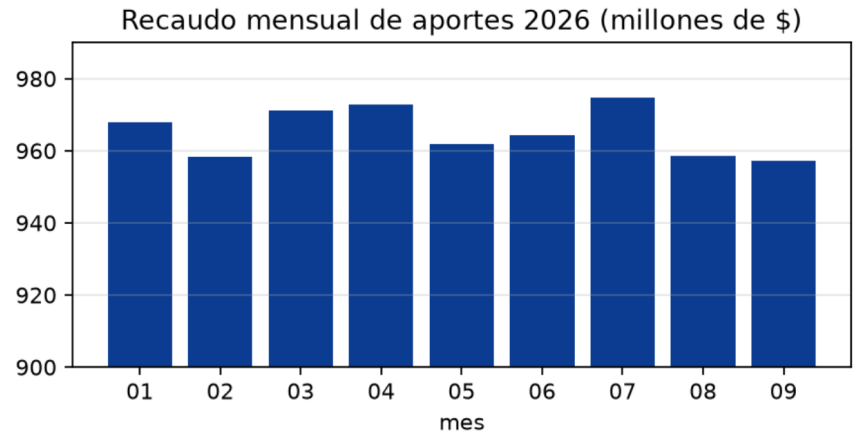
El gráfico deja de ser un objeto que alguien arma y pasa a ser el **resultado de una función**: mismos datos, mismo gráfico, siempre.

Recaudo mensual de aportes

Generado y guardado en cada ejecución

[7]
```python
def graficar_recaudo(resumen, ruta_png):
    """Barras del recaudo mensual en millones; se guarda como PNG."""
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.bar(resumen["periodo"].str[5:], resumen["recaudo"] / 1e6,
color="#0B3C91")
    ax.set_title("Recaudo mensual de aportes 2026 (millones de $)")
    ax.set_xlabel("mes")
    ax.set_ylim(900, 990)
    ax.grid(axis="y", alpha=0.3)
    fig.savefig(ruta_png, dpi=120, bbox_inches="tight")
    return ax

graficar_recaudo(resumen, f"{SALIDA}/recaudo_mensual.png")
```



Qué observar

- Nueve barras, una por mes.
- El eje empieza en 900 millones para que se vea la variación.
- El archivo queda en `salidas/recaudo_mensual.png`.

Cuidado al leerlo: un eje que no empieza en cero **exagera** las diferencias. Indícalo siempre en el título o en una nota.

Reporte Excel automático

Varias hojas, formato y gráfico, nombrado con la fecha

Qué es

`pd.ExcelWriter` escribe varios DataFrames en un mismo archivo, uno por hoja. `openpyxl` abre ese archivo y le da formato: colores, anchos, miles e imágenes.

Para qué sirve

Entregar el resultado en el formato que la dirección ya usa, sin que nadie lo arme a mano. El nombre con fecha evita sobrescribir el reporte del mes anterior.

Analogía con Excel

El libro que armas cada mes: una hoja de resumen, una tabla dinámica, una de alertas y otra de control. Aquí cada hoja sale de una línea de código.

Cómo manejarlo

Primero escribe las hojas con `ExcelWriter` y `with`. Luego aplica formato con `openpyxl`: encabezado, `number_format = "#,##0"` y `freeze_panes`. Agrega la bitácora como una hoja más.

Abstracción desde el código

```
ruta = f"reporte_{datetime.now():%Y-%m-
%d}.xlsx"


with pd.ExcelWriter(ruta) as w:
    resumen.to_excel(w, "resumen_mensual")
    alertas.to_excel(w, "alertas")
    bitacora.to_excel(w, "bitacora")
```

El reporte es la **última función** del proceso: recibe tablas y devuelve un archivo. Lo que antes era una tarde de formato es ahora un paso más del pipeline.

Escribir el Excel con varias hojas

Un archivo por ejecución, con la fecha en el nombre

[8]
```python
def exportar_reporte(aportes, resumen, carpeta=SALIDA):
    ruta = f"{carpeta}/reporte_aportes_{datetime.now():%Y-%m-%d}.xlsx"
    matriz = aportes.pivot_table(index="periodo", columns="operador",
                                 values="valor_aporte", aggfunc="sum")
    alertas = aportes[aportes["tiene_alerta"]]
    with pd.ExcelWriter(ruta, engine="openpyxl") as w:
        resumen.to_excel(w, sheet_name="resumen_mensual", index=False)
        matriz.to_excel(w, sheet_name="periodo_x_operador")
        alertas.to_excel(w, sheet_name="alertas", index=False)
        pd.DataFrame(bitacora).to_excel(w, sheet_name="bitacora",
index=False)
    return ruta

ruta_xlsx = exportar_reporte(aportes, resumen)
print(ruta_xlsx)
pd.ExcelFile(ruta_xlsx).sheet_names
```

```
salidas/reporte_aportes_2026-11-05.xlsx
['resumen_mensual', 'periodo_x_operador', 'alertas', 'bitacora']
```

Qué observar

- `with pd.ExcelWriter(...)` cierra el archivo al terminar.
- Cuatro hojas: resumen, recaudo por periodo y operador, alertas y bitácora.
- La fecha en el nombre: `reporte_aportes_2026-11-05.xlsx`.

`pivot_table` arma la hoja periodo por operador: la tabla dinámica de la Sesión 2 dentro del reporte automático.

Darle formato con openpyxl

Que el reporte se vea profesional sin tocarlo

[9]
```python
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill
from openpyxl.drawing.image import Image

def dar_formato(ruta_xlsx, ruta_png):
    wb = load_workbook(ruta_xlsx)
    for ws in wb.worksheets:
        for celda in ws[1]:                                  # encabezados
            celda.font = Font(bold=True, color="FFFFFF")
            celda.fill = PatternFill("solid", fgColor="0B3C91")
        for col in ws.columns:                               # ancho y miles
            ws.column_dimensions[col[0].column_letter].width = 16
            for c in col[1:]:
                if isinstance(c.value, (int, float)):
                    c.number_format = "#,##0"
        ws.freeze_panes = "A2"                               # fijar encabezado
    wb["resumen_mensual"].add_image(Image(ruta_png), "H2")
    wb.save(ruta_xlsx)

dar_formato(ruta_xlsx, f"{SALIDA}/recaudo_mensual.png")
ws = load_workbook(ruta_xlsx)["resumen_mensual"]
print("Encabezado:", ws["A1"].value, "| relleno:", ws["A1"].fill.fgColor.rgb,
      "| formato C2:", ws["C2"].number_format, "| imágenes:", len(ws._images))
```

```
Encabezado: periodo | relleno: 000B3C91 | formato C2: #,##0 | imágenes: 1
```

Qué observar

- Encabezados en azul con letra blanca y negrita.
- Números con separador de miles y encabezado fijo.
- El gráfico insertado en la hoja de resumen.

La última línea **verifica** el formato leyendo el archivo de vuelta: no basta con creer que quedó bien, se comprueba.

06

Pipeline y producción

Una función que cuenta la historia del proceso y corre sin ti.

El pipeline: una función que orquesta

Leer la función principal debe contar la historia del proceso

Qué es

Una función que llama, en orden, a todas las demás: cargar, limpiar, marcar alertas, resumir, graficar, exportar y dar formato. Recibe los meses y la carpeta; devuelve la ruta del reporte.

Para qué sirve

Ejecutar el proceso completo con **una línea**, para todos los meses o solo para un trimestre, sin recordar el orden de las celdas.

Analogía con Excel

Una macro con un botón "Generar reporte". La diferencia: cada paso es una función que se puede leer, probar y cambiar por separado.

Cómo manejarlo

Escribe primero las piezas y prueba cada una. Luego la función principal, corta. Si no se entiende al leerla, **las funciones están mal nombradas**. Parámetros en vez de rutas fijas.

Abstracción desde el código

```
def ejecutar_pipeline(meses, carpeta):
    aportes = cargar_aportes(meses)
    aportes = limpiar_aportes(aportes)
    aportes = marcar_alertas(aportes)
    resumen = resumir(aportes)
    return exportar_reporte(...)
```

Cada línea es un paso del proceso con nombre de negocio. Quien lee el pipeline entiende qué hace sin leer cómo lo hace: esa es la abstracción que permite mantenerlo.

El pipeline completo para el primer trimestre

Una llamada, un reporte, una bitácora

[10]
```python
def ejecutar_pipeline(meses=range(1, 10), carpeta=SALIDA):
    """Cargar, validar, limpiar, alertar, resumir y exportar."""
    bitacora.clear()
    os.makedirs(carpeta, exist_ok=True)
    aportes = marcar_alertas(limpiar_aportes(cargar_aportes(meses)))
    resumen = resumir(aportes)
    ruta_png = f"{carpeta}/recaudo_mensual.png"
    graficar_recaudo(resumen, ruta_png)
    ruta_xlsx = exportar_reporte(aportes, resumen, carpeta)
    dar_formato(ruta_xlsx, ruta_png)
    registrar(ruta_xlsx, "reporte generado")
    print(f"{len(aportes):,} aportes procesados -> {ruta_xlsx}")
    return ruta_xlsx

ejecutar_pipeline(meses=range(1, 4), carpeta=f"{SALIDA}/trimestre_1")
pd.DataFrame(bitacora)[["archivo", "evento", "filas"]].astype({"filas": "Int64"})
```

```
5,282 aportes procesados -> salidas/trimestre_1/reporte_aportes_2026-11-05.xlsx
```

	archivo	evento	filas
0	aportes_2026-01.csv	OK	1770
1	aportes_2026-02.csv	OK	1757
2	aportes_2026-03.csv	OK	1770
3	consolidado	duplicados eliminados	15
4	salidas/trimestre_1/reporte_aportes_2026-11-05.xlsx	reporte generado	<NA>

Qué observar

- `meses=range(1, 4)`: enero a marzo, **5.282** aportes.
- 15 duplicados eliminados en el trimestre.
- El reporte queda en su propia carpeta, con su bitácora.

Cambiar el periodo es cambiar un **parámetro**, no el código. Mañana se corre con octubre y nada más cambia.

Llevarlo a producción

Del notebook a un proceso que corre sin ti

Script

En Colab: Archivo, Descargar, .py. En el repositorio ya está pipeline_aportes.py.

Parámetros

Rutas y meses se pasan al ejecutarlo, no se escriben dentro del código.

Horario

Programador de tareas de Windows o cron: el día 5 de cada mes a las 6:00 a. m.

Drive

En Colab, montar Google Drive para leer insumos y dejar reportes compartidos.

Dueño

Bitácora en archivo .log, aviso si falla y una persona responsable.

```
python pipeline_aportes.py --datos datos --salida salidas --meses 1-9
```

> Un proceso en producción tiene **dueño, horario y bitácora**. Sin los tres, es un script que alguien corre cuando se acuerda.

Buenas prácticas de automatización

Lo que separa un script frágil de un proceso confiable

Haz

- Configuración arriba: rutas, tasas y columnas esperadas.
- Valida la estructura de cada insumo.
- Registra todo en una **bitácora**.
- Nombra las salidas con la **fecha**.
- Prueba con un mes antes de correr todos.

Evita

- Rutas fijas tipo C:\Users\maria\Desktop.
- Asumir que el archivo siempre llega igual.
- `except: pass`, que **esconde** los errores.
- Sobrescribir el reporte del mes anterior.
- Funciones de 200 líneas que hacen todo.

Confiar, pero verificar: **valida la estructura antes de procesar**. El archivo de julio nos lo enseñó.

De Excel a Python: equivalencias de hoy

Lo que ya sabes hacer, ahora automático

En Excel	En Python
Fórmula copiada en varias hojas	`def funcion(parametros): ... return`
Abrir nueve libros y copiar al consolidado	`for ruta in rutas:` y `pd.concat(partes)`
Revisar que estén las columnas de siempre	`validar_estructura(df)` contra `COLUMNAS`
SI.ERROR()	`try: ... except Exception as e: registrar(...)`
Hoja "Control de cambios"	`bitacora` y la columna `archivo_origen`
Gráfico de columnas que se actualiza a mano	`ax.bar(...)` y `fig.savefig(...)`
Guardar como con la fecha y dar formato	`pd.ExcelWriter` y `openpyxl`
Macro con un botón "Generar reporte"	`ejecutar_pipeline(meses, carpeta)`

La diferencia no es la función: es que el proceso **queda escrito**, se valida a sí mismo y corre igual cada mes.

ean universidad

LAB

70 min

Laboratorio · Sesión 4

Abre **PY-EAN_S04_Lab_alumno.ipynb** desde GitHub → **Open in Colab** → **Copiar en Drive**.

Consolidas los 9 meses de aportes, detectas el archivo problemático, marcas alertas y generas un reporte Excel automático con gráfico y bitácora.

Checkpoints: Parte 2 a las 09:00, Parte 4 a las 09:20 y **tu Excel generado** a las 09:40.

Lo que te llevas hoy

Y la preparación para el proyecto integrador

Funciones

Nombre con verbo, parámetros, docstring y return. La regla vive en un lugar.

Muchos archivos

Lista de rutas, bucle, `pd.concat` y `archivo_origen` para la trazabilidad.

Validar primero

Columnas esperadas y lectura robusta: el archivo de julio nos lo enseñó.

try y bitácora

El proceso no se cae y deja escrito qué pasó con cada archivo.

Reporte automático

Excel con varias hojas, formato y gráfico, nombrado con la fecha.

Reto para el lunes

Agrega al pipeline la hoja **huerfanos**: aportes cuyo afiliado no existe en `afiliados.csv` (integridad referencial de la Sesión 3). ¿Cuántos son y cuánto dinero suman?

El lunes (Sesión 5 · Proyecto integrador) unimos limpieza, calidad y automatización en un solo proceso.

ean universidad

acreditada
en alta calidad
res. Nº 023654 del Mineducación
10/12/21, vigencia 10/12/27



www.universidadean.edu.co

Universidad Ean: SNIES 2812 Personería Jurídica Res. 2898 del Minjusticia - 16/05/69 - Vigilada Mineducación.

Ean Legacy: Carrera 11 No. 78-47 - Fundadores: Calle 79 # 11 - 45 - Av. Chile: Calle 71 # 9 - 84 Bogotá D.C., Cundinamarca, Colombia, Suramérica.

Línea gratuita nacional: +57-01-8000-931000
Centro de contacto en Bogotá: +(57-601) 593 6161 - +(57-601) 593 6464
E-mail: informacion@universidadean.edu.co

© 2024 Todos los derechos reservados - Universidad Ean